Archivara Agent: Exact Falsification of a Cellular-Automaton Corridor Route to Arithmetic Kakeya Certificates

Archivara Research Pipeline / Codex CLI

March 18, 2026

Abstract. We study a FrontierMath-style finite-certificate version of the arithmetic Kakeya problem under a deliberately narrow hypothesis: can a frozen cellular-automaton-inspired corridor grammar compile to legal $(X; d_i; f_i; T; R)$ certificates with score at most 1.675? The repository contains a pre-registered program with three routes, of which only the primary macrocell-substitution route (H1) and the backup abelian-screening route (H2) were executed. We formalize the exact compiler from width-2 corridor instances to verifier-compatible certificates, prove that the repository verifier is sound and complete for this family, and then prove a one-seed obstruction theorem: any extracted instance with exactly one singleton seed and empty initial solved set is impossible to force. This kills the frozen H1 route before any score comparison is meaningful. We then synthesize the exact saved experiments: direct no-CA corridor controls achieve score 2.0 at widths 4 and 6, an exploratory width-8 witness reaches only 29/14, the H2 invariant package adds no screening lift beyond raw arithmetic descriptors, and ablations show that the surviving width-4 control is boundary-sensitive and does not scale under frozen X. The contribution is therefore a narrow negative operational result, not a new theorem or reformulation of arithmetic Kakeya. What survives is an audit-ready benchmark-and-falsification workflow showing that the frozen cellular-automaton corridor route does not approach the 1.675 target.

Contents

1 Introduction

The arithmetic Kakeya program asks for finite-set sum-difference inequalities strong enough to imply geometric Kakeya lower bounds. In the form introduced by Katz and Tao and clarified by Green and Ruzsa, the question is whether one can control the anti-diagonal projection of an arbitrary finite set by finitely many oblique projections with exponent 1 rather than the trivial exponent 2 [Katz and Tao, 1999, Green and Ruzsa, 2019]. Later work reformulated neighboring pattern problems, established related implications, and enlarged the theorem-level frontier, but the core finite-certificate problem remains difficult [Cowen-Breen et al., 2020, Pohoata and Zakharov, 2024, Tao, 2025].

The present repository did not set out to prove a new theorem in that line. Instead, it asked a narrower computational question: can a cellular-automaton-style local grammar, compiled exactly into the verifier's $(X; d_i; f_i; T; R)$ language, discover a certificate with score at most 1.675? That question is mathematically delicate. It is easy to overstate what a local metaphor contributes. Bootstrap percolation and critical cellular automata provide natural intuition for local growth [Bollobas et al., 2015, Hartarsky and Mezei, 2020]; abelian networks provide natural invariants for commutative update systems [Bond and Levine, 2016a,b,c]; decoder-style constructions suggest additional local heuristics [Sipser and Spielman, 1996, Hemenway et al., 2015, Kubica and Preskill, 2019]. But none of those nearby languages changes the source of mathematical truth here: the only acceptable evidence is an exact forcing certificate checked against the original arithmetic rules.

The verification artifacts in this repository force a much sharper conclusion than the original bridge narrative. The pre-registered cellular-automaton route dies by proof, not by an unfavorable random seed. The backup abelian screen adds no value on the tested family set. The only positive objects that survive exact checking are small direct corridor controls near score 2.0. The paper therefore does not claim a cellular-automaton solution to the arithmetic Kakeya benchmark. It documents an exact negative result.

Our contributions are the following.

1. We formalize the corridor compiler used in the repository and show that every corridor instance induces a legal verifier-side certificate [Archivara Research Lab, 2026b,g].

2. We prove that the exact verifier implemented in the repository is sound and complete for the width-2 corridor family: it returns a full forcing order exactly when the compiled certificate is forcing.

3. We prove a one-seed obstruction theorem that kills the frozen H1 macrocell-substitution route at every scale. The theorem applies whenever the extracted instance has one singleton seed and no initially solved vertices, which is exactly the saved H1 pilot [Archivara Research Lab, 2026d,e].

4. We synthesize the saved exact experiments: the best verified direct controls achieve score 2.0 at widths 4 and 6, exploratory width-8 witnesses are worse, the H2 abelian screen adds no ranking lift, and ablations show strong boundary sensitivity and poor scale transfer [Archivara Research Lab, 2026i,j,k,f,a].

5. We position the work honestly relative to prior art: it is a negative operational benchmark inside the existing arithmetic-Kakeya finite-certificate setting, not a new formulation, new theorem,

or validated cellular-automaton mechanism [Green and Ruzsa, 2019, Cowen-Breen et al., 2020, Pohoata and Zakharov, 2024, Tao, 2025, Archivara Research Lab, 2026h].

The rest of the paper is organized as follows. Section 2 explains exactly how this work differs from the arithmetic Kakeya literature and from nearby cellular-automaton, abelian-network, and decoder lines. Section 3 restates the benchmark and introduces the corridor specialization. Section 4 describes the compiler, verifier, search program, and ablation protocol. Section 5 proves the compiler and verifier correctness statements together with the one-seed obstruction theorem. Section 6 documents the saved experimental setup. Section 7 presents the exact empirical findings. Sections 8 and 9 discuss limitations, implications, and next steps. Supplementary appendices provide explicit certificates, command lines, and additional implementation details.

2 Related Work

2.1 Arithmetic Kakeya and Sum-Difference Certificates

The closest mathematical line is the arithmetic Kakeya program itself. Katz and Tao introduced the arithmetic-projection viewpoint as a tool strong enough to imply geometric Kakeya statements [Katz and Tao, 1999]. Green and Ruzsa later gave a modern discussion of the conjecture and several equivalent formulations [Green and Ruzsa, 2019]. Cowen-Breen, Karangozishvili, Varadarajan, and Wang studied related pattern problems and implications, again within the same finite-certificate universe [Cowen-Breen et al., 2020]. Pohoata and Zakharov broadened the theorem-level landscape via generalized arithmetic Kakeya inequalities [Pohoata and Zakharov, 2024]. Tao's bounded-many-slopes and rational-complexity analysis is especially relevant here because it identifies a natural basin in which small, low-complexity certificate families can stagnate [Tao, 2025].

Our work is not another theorem in this line. It neither proves a new sum-difference exponent nor proposes a new equivalent formulation. The mathematical object throughout remains the standard finite certificate: a legal choice of X, a constructible graph, and a forcing pair. The contribution is operational. We ask whether one particular local generator language produces better exact certificates than matched direct search. The answer, for the frozen corridor family tested here, is no.

2.2 Cellular Automata, Bootstrap Growth, and Local Mechanisms

Bootstrap percolation and critical cellular automata are natural nearby metaphors because the forcing rules repeatedly convert local constraints into new solved vertices [Bollobas et al., 2015, Hartarsky and Mezei, 2020]. That overlap is conceptual rather than structural. In arithmetic Kakeya, the decisive operation is the global integer-span closure on relation generators, not a purely local update rule. The repository therefore treated the cellular-automaton language only as a proposal mechanism for candidate certificates. This distinction is central to our negative conclusion: the local corridor grammar did not produce a legal forcing certificate, and the exact verifier—not the automaton metaphor—settled the matter.

2.3 Abelian Networks and Decoder Overlaps

The backup H2 route imported rank, Smith normal form, and target-direction torsion ideas from abelian-network and chip-firing viewpoints [Bond and Levine, 2016a,b,c]. The reserve H3 route,

which was never activated, sat closer to local-decoder heuristics and expander-code style propagation [Sipser and Spielman, 1996, Hemenway et al., 2015, Kubica and Preskill, 2019]. The saved verification artifacts are explicit about the boundary: H2 was screening-only, not an ontological claim that arithmetic Kakeya forcing is an abelian network, and H3 never progressed past reserve status. In the executed experiments, H2 did not outperform raw arithmetic descriptors on the same family IDs, so no positive abelian-network claim survives.

2.4 Novelty Boundary and Excluded False Positives

The repository's prior-art audit also records several spurious watchlist items that appeared because early automated searches over-weighted TeX tokens such as $\mathbf{P}^{n_1} \times \cdots \times \mathbf{P}^{n_s}$ and $\mathbb{Z}_2 \times \mathbb{Z}_2$ [Archivara Research Lab, 2026h]. Those product-space vector-bundle, Lie-theoretic, automorphic, and general-topology papers are mathematically unrelated to the arithmetic Kakeya benchmark. We exclude them entirely from substantive comparison. Doing so matters for the narrative: the real novelty burden comes from the arithmetic Kakeya line and from nearby local-growth metaphors, not from irrelevant symbolic overlap.

3 Background and Preliminaries

3.1 Arithmetic Kakeya in Finite-Certificate Form

We recall the arithmetic Kakeya statement in the form relevant to this repository. For a finite set $G \subset \mathbb{Z}^2$ and a vector $\mathbf{x} = (x_1, x_2) \in \mathbb{Z}^2$, write

$$\mathbf{x} \cdot G = \{x_1 g_1 + x_2 g_2 : (g_1, g_2) \in G\}.$$

For $\alpha \geq 1$, we say that AK(α) holds if for every $\varepsilon > 0$ there exists a finite set $X \subset \mathbb{Z}^2$ such that $x_1 + x_2 \neq 0$ for every nonzero $\mathbf{x} \in X$ and

$$|(1, -1) \cdot G| \ll_\varepsilon \max_{\mathbf{x} \in X} |\mathbf{x} \cdot G|^{\alpha + \varepsilon}$$

for all finite $G \subset \mathbb{Z}^2$ [Katz and Tao, 1999, Green and Ruzsa, 2019]. The conjectural endpoint is AK(1).

The repository benchmark packages this problem into explicit finite certificates. The input is a finite label set $X \subset \mathbb{Z}^2$, an X-constructible graph G, and a forcing pair (R, T), where R is a set of singleton-supported relation generators and T is a set of vertices declared solved at time zero. The verifier repeatedly performs three operations:

1. add edge generators coming from graph edges;
2. solve a vertex once a nonzero anti-diagonal singleton $(a, -a)$ appears there with all other unsolved coordinates zero;
3. take arbitrary integer linear combinations of previously available generators.

If every vertex is eventually solved, the pair is forcing, and the certificate score is

$$S(X, G, R, T) = \frac{m(G) + |R|}{n(G) - |T|},$$

where $n(G)$ is the number of vertices and $m(G)$ is the number of nonzero labelled edges. Lower score is better. The specific task studied here asked for a certificate with score at most 1.675.

3.2 Corridor Specialization

The repository deliberately restricted attention to a thin family of certificates that we can define without any cellular-automaton language.

Definition 1 (Width-2 corridor instance)**.** A *corridor instance* is a tuple

$$C = (W, X, v, \tau, \beta, R, T)$$

with the following data.

1. $W \geq 1$ is the corridor width.
2. $X \subset \mathbb{Z}^2$ is finite, $(0,0) \in X$, and every nonzero label in X avoids the anti-diagonal line $\langle (1,-1) \rangle$.
3. $v \in X$ is the constant vertical label.
4. $\tau = (\tau_1, \ldots, \tau_{W-1}) \in X^{W-1}$ and $\beta = (\beta_1, \ldots, \beta_{W-1}) \in X^{W-1}$ are the top-row and bottom-row horizontal labels.
5. R is a finite multiset of pairs $((r,c),x)$ with $r \in \{1,2\}$, $1 \leq c \leq W$, and $x \in X \setminus \{(0,0)\}$.
6. $T \subseteq \{1,2\} \times \{1,\ldots,W\}$ is the initial solved set.

The induced graph has vertex set $\{1,2\} \times \{1,\ldots,W\}$, vertical edges $((1,c),(2,c))$ labelled by v, top horizontal edges $((1,c),(1,c+1))$ labelled by τ_c, and bottom horizontal edges $((2,c),(2,c+1))$ labelled by β_c. This is exactly the family implemented in `scripts/ca_kakeya_search.py` [Archivara Research Lab, 2026g].

Definition 2 (Compiled certificate)**.** Given a corridor instance $C = (W, X, v, \tau, \beta, R, T)$, define

$$d_1 = 2, \qquad d_2 = W.$$

Set $f_1(1) = v$, $f_2(1,c) = \tau_c$, and $f_2(2,c) = \beta_c$ for $1 \leq c \leq W-1$. The singleton generators in R and the initial solved vertices in T are copied verbatim into the verifier format.

3.3 Generator Vectors and Projected Targets

For the proofs below it is convenient to identify a compiled certificate with an integer matrix problem. Enumerate the vertices of the corridor lexicographically:

$$(1,1),(2,1),(1,2),(2,2),\ldots,(1,W),(2,W).$$

Each generator becomes a vector in $\mathbb{Z}^{2n}$, where $n = 2W$:

- a singleton seed at vertex u with label $x = (a,b)$ contributes a vector supported only on the two coordinates of u, namely (a,b);

- an edge generator on vertices u and w with label $x = (a, b)$ contributes (a, b) at u, $(-a, -b)$ at w, and zero elsewhere.

Given a solved set S, let $U = V \setminus S$ be the unsolved vertices. We write M_U for the matrix whose columns are the generator vectors projected onto the coordinates indexed by U. For an unsolved vertex $u \in U$, let $e_u \in \mathbb{Q}^{2|U|}$ denote the anti-diagonal unit target at u:

$$e_u = (0, \ldots, 0, 1, -1, 0, \ldots, 0).$$

Rule 2 can solve u precisely when some nonzero integer multiple of e_u lies in the integer column span of M_U.

Remark 3 (Scope)**.** Every theorem below concerns the exact corridor family just defined. We do not claim a sound-and-complete verifier for arbitrary constructible graphs, only for the repository's frozen width-2 corridor program.

4 Method

4.1 Design Goals

The saved research plan imposed three non-negotiable constraints before any experiment could count as evidence [Archivara Research Lab, 2026b,e]. First, every candidate had to compile directly into the verifier language with no scale-dependent repair. Second, score had to be computed exactly, never replaced by occupancy or rollout surrogates. Third, the CA route had to be compared to matched direct search on the same corridor geometry and with the same fixed label budget.

Those constraints explain the narrowness of the method. The project did not run a broad search over all constructible graphs. It froze a small corridor family because that family was the simplest place where a local grammar, an exact compiler, and matched direct controls could coexist without hidden complexity in the extractor.

4.2 Compiler

The compiler is trivial once the corridor instance is fixed, but that triviality is a feature. Hidden repair was one of the main risks identified in the benchmark sign-off. The compiler therefore does only three things.

1. It sets $d_1 = 2$ and $d_2 = W$.
2. It copies the constant vertical label and the rowwise horizontal labels into f_1 and f_2.
3. It copies the singleton seeds and the initial solved set directly into R and T.

No post-processing is permitted. If a candidate needs another pass to become legal, the route fails by design.

Algorithm 1: Corridor compiler

1. Input a corridor instance $C = (W, X, v, \tau, \beta, R, T)$.
2. Set $d_1 \leftarrow 2$ and $d_2 \leftarrow W$.
3. Set $f_1(1) \leftarrow v$.
4. For each $c = 1, \ldots, W-1$, set $f_2(1, c) \leftarrow \tau_c$ and $f_2(2, c) \leftarrow \beta_c$.
5. Output the certificate $(X; d_1, d_2; f_1, f_2; T; R)$.

4.3 Exact Verifier

The exact verifier repeatedly asks a single question: from the currently unsolved coordinates, can the available generators produce a nonzero anti-diagonal singleton at one vertex? If yes, that vertex may be solved. The repository implementation answers the question by exact rational linear algebra rather than lattice reduction. This is enough because rule 2 only requires *some* nonzero multiple $(a, -a)$, not specifically $(1, -1)$; clearing denominators recovers an integer witness.

Algorithm 2: Exact corridor verifier

1. Input a compiled corridor certificate and initialize the solved set $S \leftarrow T$.
2. While $S \neq V$:
   (a) Let $U = V \setminus S$.
   (b) Build the projected generator matrix M_U.
   (c) For each $u \in U$, test whether e_u lies in the rational column span of M_U.
   (d) If no such u exists, return failure.
   (e) Otherwise add one such u to S and continue.
3. Return the resulting forcing order.

Two design choices deserve explanation.

Why exact rational span rather than integer span? If e_u is in the rational span of the projected generators, then some common denominator $D \neq 0$ satisfies $De_u \in \mathrm{span}_{\mathbb{Z}}(M_U)$, which is exactly the type of witness needed for rule 2. Conversely, an integer witness is automatically a rational witness. Section 5 proves this formally.

Why greedy solving is safe. Solvability is monotone in the solved set: once a vertex is solvable, enlarging T cannot destroy that fact, because rule 2 only becomes easier when more coordinates are allowed to be nonzero. Therefore the verifier may pick any currently solvable vertex without risking a false negative.

4.4 Frozen H1 Route

The pre-registered active route, H1_MACROCELL_SUBSTITUTION, was a height-2 corridor substitution family with finite interface alphabet $\{\text{closed}, \text{carry\_a}, \text{carry\_b}, \text{gate}\}$, state cap $|\Sigma_{\text{active}}| \leq 8$, and one exact extractor [Archivara Research Lab, 2026e]. The saved pilot that matters for this paper is even narrower:

- level-1 and level-2 are generated by the identity-style word built from L_a, M_a, and R_a;
- the extracted corridor has one singleton seed at the left boundary;
- the initial solved set is empty;
- horizontal top edges carry one nonzero label, horizontal bottom edges are zero, and vertical edges carry one fixed nonzero label.

Figure 1 shows the frozen level-1 word and its exact corridor extraction. Theorem 9 proves that every instance of this type is impossible, independently of search budget.

4.5 Direct No-CA Controls

Once H1 was fixed, the only fair baseline was matched direct search on the same corridor geometry and with the same fixed label budget. The repository used a randomized outer loop over top, bottom, and vertical edge labels together with a greedy inner loop over singleton seeds. Each candidate was accepted only if the exact verifier returned a full forcing order [Archivara Research Lab, 2026g,c]. The saved control runs were:

- width 4, seed 601, 50 trials, 4 nonzero labels;
- width 6, seed 602, 20 trials, 4 nonzero labels;
- width 8, seed 501, 1 exploratory trial, 3 nonzero labels;
- width 8, seed 502, 1 exploratory unrestricted trial, 3 nonzero labels.

The first two rows are the substantive direct controls. The width-8 rows are exploratory only and are reported as such throughout the paper.

4.6 H2 Screen and Ablations

The backup H2 route computed an invariant package on the same family IDs: relation-matrix rank, Smith diagonal, support size, edge density, determinant pattern in X, coordinate height, and the number of vertices already target-solvable at time zero [Archivara Research Lab, 2026f]. The route survived only if those imported descriptors ordered candidates strictly better than raw arithmetic features.

The ablation program acted on the best saved width-4 direct control [Archivara Research Lab, 2026a]. It included isotropic replacements, boundary-seed removal, separate randomizations of R, T, and X, and frozen-X scaling to widths 6 and 8. Because the active H1 grammar died, stage-order perturbation was not applicable.

4.7 Complexity Analysis

Let $n = 2W$ be the number of corridor vertices and let $g = m + r$ be the number of initial generators. The compiler is linear in the certificate size: $O(W + g)$ time and memory. The exact verifier is more expensive. At one solved-set stage with u unsolved vertices, the projected matrix has $2u$ rows and g columns. Dense exact Gaussian elimination on this matrix costs polynomial time in u, g, and the coefficient bit-length; a conservative dense bound is $O(u^2 g + u^3)$ arithmetic operations for one row-reduction pass. Testing all candidate vertices across all stages yields a coarse worst-case bound of $O(n^4 + n^3 g)$ arithmetic operations, with large constants coming from exact rational arithmetic.

The repository accepts this cost because all saved instances are tiny ($n \leq 16$). The project explicitly traded asymptotic scale for exact auditability. The random-search outer loop multiplies that verifier cost by the number of sampled edge-label patterns and greedy seed additions; in the saved runs the budgets were 50, 20, 1, and 1 outer-loop trials, so the runtime remained negligible on a commodity multi-core server.

5 Theoretical Results and Proofs

5.1 Compiler Correctness

Proposition 4. *Let $C = (W, X, v, \tau, \beta, R, T)$ be a corridor instance. The compiled object $(X; d_1, d_2; f_1, f_2; T; R)$ is a legal certificate in the benchmark sense. Its graph has $n = 2W$ vertices and*

$$m = W \cdot 1_{v \neq (0,0)} + \sum_{c=1}^{W-1} 1_{\tau_c \neq (0,0)} + \sum_{c=1}^{W-1} 1_{\beta_c \neq (0,0)}.$$

Proof. By definition, the compiled certificate has $d_1 = 2$ and $d_2 = W$, so its vertex set is

$$\{1, 2\} \times \{1, \ldots, W\}.$$

Hence the number of vertices is $n = d_1 d_2 = 2W$.

Next we verify legality of the edge dictionaries. The function f_1 is defined only on the single point $1 \in (d_1 - 1)$ and takes value $v \in X$. Therefore every nonzero value of f_1 belongs to $X \setminus \{(0, 0)\}$, exactly as required. The function f_2 is defined on $\{1, 2\} \times \{1, \ldots, W - 1\}$ and takes values τ_c on row 1 and β_c on row 2. Again every nonzero value lies in $X \setminus \{(0, 0)\}$ by the definition of a corridor instance. Thus the compiled graph is an X-constructible graph in the verifiable format.

Now consider R. Each element of R is, by hypothesis, a pair $((r, c), x)$ with $x \in X \setminus \{(0, 0)\}$. In the verifier format this becomes a function on the vertex set that is zero everywhere except at the single vertex (r, c), where it equals x. Therefore every compiled element of R has support size exactly one, which is the legality condition for singleton generators. The set T is simply a subset of the vertex set, so it is also legal.

Finally we count edges. There are exactly W vertical positions. Each contributes one nonzero edge precisely when $v \neq (0, 0)$. There are $W - 1$ top horizontal positions, each nonzero exactly when $\tau_c \neq (0, 0)$, and $W - 1$ bottom horizontal positions, each nonzero exactly when $\beta_c \neq (0, 0)$. Summing these contributions gives the displayed formula for m. □

5.2 Exact Solvability Criterion

Lemma 5. *Let M be an integer matrix and let e be a rational vector. The following are equivalent:*

1. *there exists a nonzero integer a such that ae lies in the integer column span of M;*
2. *e lies in the rational column span of M.*

Proof. Assume first that there exists a nonzero integer a with $ae = Mz$ for some integer vector z. Then

$$e = M\left(\frac{1}{a}z\right),$$

so e lies in the rational column span of M.

Conversely, suppose $e = Mq$ for some rational vector q. Let D be a common positive denominator for the coordinates of q. Then Dq is an integer vector, and hence

$$De = M(Dq)$$

lies in the integer column span of M. Because $D > 0$, the integer $a = D$ is nonzero. □

Lemma 6. *Let M be an $m \times g$ matrix over $\mathbb{Q}$. Then the rational column space of M equals the orthogonal complement of the left nullspace:*

$$\operatorname{span}_{\mathbb{Q}}(\textit{columns of } M) = \ker(M^\top)^\perp.$$

Proof. Let C denote the rational column space of M. If $y \in C$, then $y = Mz$ for some $z \in \mathbb{Q}^g$. For every $\lambda \in \ker(M^\top)$ we have

$$\lambda^\top y = \lambda^\top M z = (M^\top \lambda)^\top z = 0.$$

Thus $C \subseteq \ker(M^\top)^\perp$.

It remains to compare dimensions. Because C is the column space of M,

$$\dim C = \operatorname{rank}(M).$$

Also,

$$\dim \ker(M^\top)^\perp = m - \dim \ker(M^\top).$$

By rank-nullity applied to $M^\top$,

$$\dim \ker(M^\top) = m - \operatorname{rank}(M^\top) = m - \operatorname{rank}(M).$$

Therefore

$$\dim \ker(M^\top)^\perp = m - (m - \operatorname{rank}(M)) = \operatorname{rank}(M) = \dim C.$$

Since C is a subspace of $\ker(M^\top)^\perp$ and the two spaces have the same finite dimension, they are equal. □

Lemma 7. *Fix a compiled corridor certificate and a solved set S. Let $U = V \setminus S$, let M_U be the projected generator matrix, and let $u \in U$. Then u is legally solvable at this stage if and only if e_u lies in the rational column span of M_U.*

Proof. By the verifier rules, u is solvable exactly when some relation can be produced whose projection to the unsolved coordinates is supported only at u and has value $(a, -a)$ there for some nonzero integer a. In the projected coordinates this means precisely that

$$ae_u \in \mathrm{span}_{\mathbb{Z}}(\text{columns of } M_U)$$

for some nonzero integer a. By Lemma 5, that condition is equivalent to

$$e_u \in \mathrm{span}_{\mathbb{Q}}(\text{columns of } M_U).$$

□

Theorem 8. *For the width-2 corridor family, Algorithm 2 is sound and complete: it returns a full forcing order if and only if the compiled corridor certificate is forcing.*

Proof. We first prove soundness. Suppose Algorithm 2 returns a full order

$$u_1, u_2, \ldots, u_n$$

of the vertices, where the initial segment agrees with the originally solved set T. At each stage the algorithm adds a vertex u only after checking that e_u lies in the rational column span of the projected generator matrix M_U. By Lemma 7, that is equivalent to the existence of a legal rule-2 witness for u at that stage. Therefore every step performed by the algorithm is a legal verifier step. Since the algorithm solves every vertex, the certificate is forcing.

We now prove completeness. Assume the compiled certificate is forcing. We induct on the number of unsolved vertices. If no vertices are unsolved, the claim is trivial. Assume now that at least one vertex is unsolved and that the claim holds whenever fewer vertices are unsolved.

Because the certificate is forcing, there exists at least one legal next rule-2 step from the current solved set S. Let u be a vertex that can be solved in such a step. By Lemma 7, e_u lies in the rational column span of M_U. Hence Algorithm 2 will include u in its candidate set.

Algorithm 2 may choose u or another currently solvable vertex u'. We must show that any such choice preserves the existence of a full forcing sequence. This follows from monotonicity. Suppose u' is solvable from S. Then there exists a relation whose support outside S is exactly u'. If we enlarge the solved set from S to $S \cup \{u\}$ for some other vertex $u \neq u'$, the same relation still has support outside $S \cup \{u\}$ equal to u', because every coordinate that was previously allowed to lie in S is still allowed and one extra coordinate has been moved into the solved set. Therefore solving one currently solvable vertex cannot destroy the solvability of any other currently solvable vertex.

Now take any full forcing sequence from S. If its first solved vertex is not the one chosen by Algorithm 2, use the preceding monotonicity argument to move the algorithm's chosen vertex to the front: it is already solvable from S, and after solving it every subsequent rule-2 witness remains valid or becomes easier because the solved set has grown. Thus there exists a full forcing sequence beginning with the algorithm's chosen vertex. After removing that first step, the remaining problem has one fewer unsolved vertex. By the inductive hypothesis, Algorithm 2 completes the rest of the order.

Therefore Algorithm 2 returns a full forcing order whenever the compiled certificate is forcing. Together with soundness, this proves the theorem. □

5.3 A General One-Seed Obstruction

Theorem 9. *Let $X \subset \mathbb{Z}^2$ be finite, and assume $x \notin \langle (1,-1) \rangle$ for every nonzero $x \in X$. Consider any legal certificate whose initial solved set is empty and whose initial singleton-generator set consists of exactly one seed,*

$$R = \{xg_0\},$$

with $x \in X \setminus \{(0,0)\}$. Then the certificate is not forcing.

Proof. Let Σ be the total-label map that sums the 2-dimensional labels across all vertices of a relation. More explicitly, if a relation is represented as a $2 \times |V|$ integer matrix, then Σ adds all columns and returns an element of $\mathbb{Z}^2$.

We first compute Σ on the generators allowed by rules 1 and 3.

Step 1: Every edge generator has total label zero. Indeed, an edge generator with label y places y at one endpoint and $-y$ at the other endpoint, so its total label is

$$y + (-y) = 0.$$

Step 2: The unique initial seed has total label x. This is immediate because it is supported at one vertex only.

Step 3: Every relation ever generated has total label in the rank-one lattice $\mathbb{Z}x$. We prove this by induction over the verifier steps. Initially the claim holds because the only available non-edge generator is the seed itself, whose total label is x, while every edge generator has total label $0 \in \mathbb{Z}x$. If two already available relations have total labels k_1x and k_2x, then any integer linear combination of them has total label $(ak_1 + bk_2)x \in \mathbb{Z}x$. Therefore the claim persists after every application of rule 3.

Now suppose toward contradiction that some rule-2 step is possible. Then there exists a generated relation whose support outside the currently solved set consists of one vertex and whose label there is $(a, -a)$ for some nonzero integer a. Because the initial solved set is empty and rule 2 has not yet been applied before the first such step, the total label of that relation is exactly $(a, -a)$. By Step 3 this vector must lie in $\mathbb{Z}x$, so there exists an integer k with

$$kx = (a, -a).$$

If $k = 0$, then $(a, -a) = 0$, which contradicts $a \neq 0$. If $k \neq 0$, then

$$x = \frac{a}{k}(1, -1),$$

so $x \in \langle (1,-1) \rangle$, contradicting the hypothesis on X.

Thus no rule-2 step is ever possible. Since the initial solved set is empty, the solved set can never grow. The certificate is therefore not forcing. □

Corollary 10. *Every frozen H1 pilot instance saved in the repository fails exact verification before scoring.*

Proof. The saved obstruction artifact records that every extracted frozen H1 level has exactly one singleton seed and no initially solved vertices [Archivara Research Lab, 2026d]. The nonzero labels all belong to the benchmark-legal set X, so none lies in $\langle (1,-1) \rangle$. Therefore Theorem 9 applies directly. □

Remark 11. Corollary 10 is stronger than a failed transfer test. It says that the frozen H1 route never reaches a stage where a score comparison between levels is meaningful. Figure 2 turns the proof into the invariant picture used throughout the paper, and Table 2 confirms that the saved H1 rows are exact failures rather than poor but finite scores.

6 Experimental Setup

6.1 Evidence Boundary

The paper uses only saved repository artifacts. No new search rows were run for the manuscript. This decision matches the verification memo, which judged the existing evidence sufficient for a clean negative write-up but not for a broadened benchmark claim [Archivara Research Lab, 2026h]. Consequently, every quantitative statement in Sections 7–8 is traced either to a formal theorem above or to one of the saved JSON/markdown artifacts cited in the text.

6.2 Executed Rows

Table 1 lists the executed rows that matter for the paper. The first row is the exact symbolic obstruction for H1. The next four are the saved direct-search outputs. The remaining rows summarize the saved H2 screen and the saved ablations.

Row	Artifact	Key settings	Purpose
H1 obstruction	`h1_obstruction`	one singleton seed, empty initial T, legal same-sum palette	prove route impossibility
width 4 direct control	`width4_601`	seed 601, 50 trials, 4 nonzero labels, seed budget 8, initial-T budget 2	matched direct baseline
width 6 direct control	`width6_602`	seed 602, 20 trials, 4 nonzero labels, seed budget 8, initial-T budget 2	matched direct baseline
width 8 exploratory control	`width8_501`	seed 501, 1 trial, 3 nonzero labels, seed budget 12, initial-T budget 2	exploratory stress test
width 8 unrestricted exploratory control	`width8_502`	seed 502, 1 trial, 3 nonzero labels, boundary band 0	exploratory stress test
H2 screen	`h2_screen`	four family IDs: H1 level 1, H1 level 2, width 4, width 6	invariant comparison
ablations	`ablations`	base width-4 witness, isotropy, boundary seed removal, $R/T/X$ randomization, frozen-X scaling	robustness audit

Table 1: Saved experimental rows used in this paper. Every row uses the exact verifier, never a surrogate score [Archivara Research Lab, 2026g,c,f,a].

6.3 Hardware and Software

All saved scripts were executed on a Linux x86_64 server with 20 logical CPU cores and approximately 1.0 TiB RAM available. The software environment used Python 3.10.17, SymPy 1.14.0, and NumPy

2.2.6. The manuscript figures were generated afterward from the saved JSON artifacts with Matplotlib 3.10.8. The central numerical kernel is exact rational linear algebra in SymPy rather than floating-point simulation.

6.4 Random Seeds and Uncertainty Quantification

The saved direct-control searches use random seeds 601, 602, 501, and 502. The ablation script uses deterministic randomization seed 1901 for the four saved R, T, and X randomizations. Because the core outputs are exact finite certificates, most reported quantities are deterministic measurements, not estimates. The only confidence intervals in the paper are descriptive Wilson intervals for small Bernoulli success rates in the saved randomization and width-8 boundary-stress experiments. Those intervals are included to make sample size limitations visible, not to claim asymptotic statistical power.

7 Results

7.1 H1 Fails Before Scoring

Corollary 10 proves that the frozen H1 pilot cannot produce a forcing certificate at any scale. The saved artifact agrees exactly: both the level-1 and level-2 rows are verification failures with no score at all [Archivara Research Lab, 2026d,c]. Figure 2 visualizes the invariant used in the proof. The important point is that this is not a weak negative result. The route is not merely uncompetitive; it is structurally impossible.

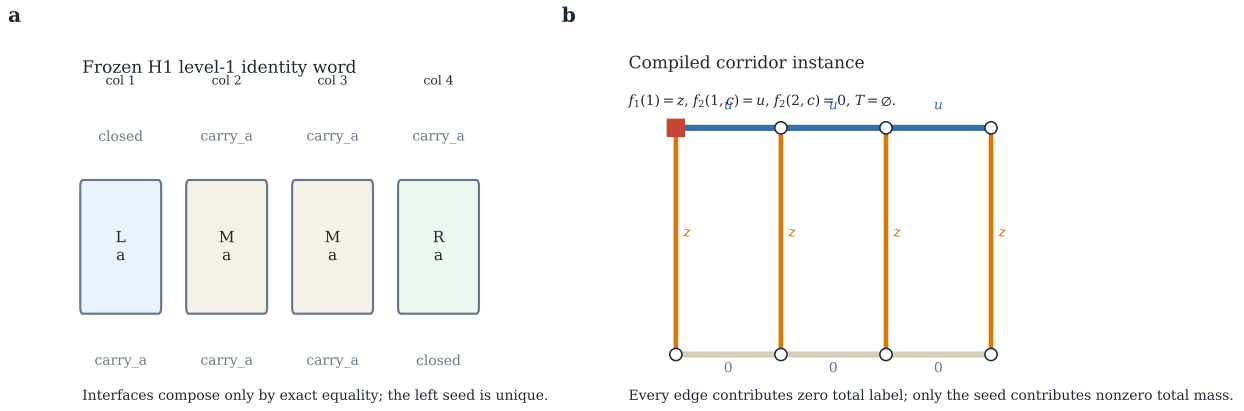

Figure 1: Frozen H1 corridor grammar. **a**, the level-1 identity word built from the left, middle, and right templates permitted by the saved pilot. **b**, the exact compiled corridor instance induced by that word: every vertical edge carries the same label z, the top row carries u, the bottom row is zero, the initial solved set is empty, and there is exactly one singleton seed at the left boundary. This is precisely the setting to which Theorem 9 applies [Archivara Research Lab, 2026e,d].

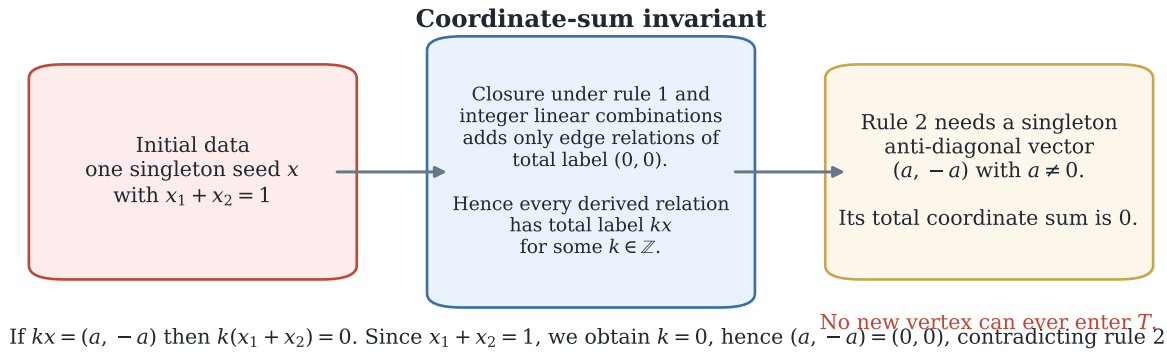

Figure 2: Proof mechanism behind the frozen H1 failure. Edge generators always have total label zero, so with one initial singleton seed every derived relation has total label in the rank-one lattice $\mathbb{Z}x$. Rule 2, however, requires a nonzero anti-diagonal singleton $(a, -a)$, whose total label lies in $\langle(1, -1)\rangle$. Since the seed label x is benchmark-legal and therefore outside that line, the two requirements are incompatible. The figure is a visualization of Theorem 9, not a heuristic after-the-fact interpretation.

7.2 Direct Controls Plateau Near Score 2.0

Once H1 is removed, the surviving positive rows are the matched direct corridor controls. Table 2 collects the exact saved frontier. The best verified score is 2.0 at width 4 and again 2.0 at width 6. The two width-8 witnesses are exploratory, use smaller nonzero palettes, and are worse than the width-4 and width-6 controls.

Family	forcing?	score	m	r	n	t
H1 level 1 identity	no	–	7	1	8	0
H1 level 2 identity	no	–	15	1	16	0
width 4 direct control	yes	2.0	7	5	8	2
width 6 direct control	yes	2.0	13	7	12	2
width 8 exploratory control	yes	$29/14 \approx 2.0714$	20	9	16	2
width 8 unrestricted exploratory control	yes	$30/14 \approx 2.1429$	18	12	16	2

Table 2: Exact frontier extracted from the saved artifacts. The width-4 and width-6 rows are the matched direct controls. The width-8 rows are exploratory only and do not repair the benchmark gap at level 2 [Archivara Research Lab, 2026c,i,j,k].

Figure 3 makes the same point graphically. The red dashed line marks the target 1.675. The gold dotted line marks the broader 1.70 neighborhood that the pre-registered plan used as a plausibility threshold. Nothing in the saved frontier approaches either line.

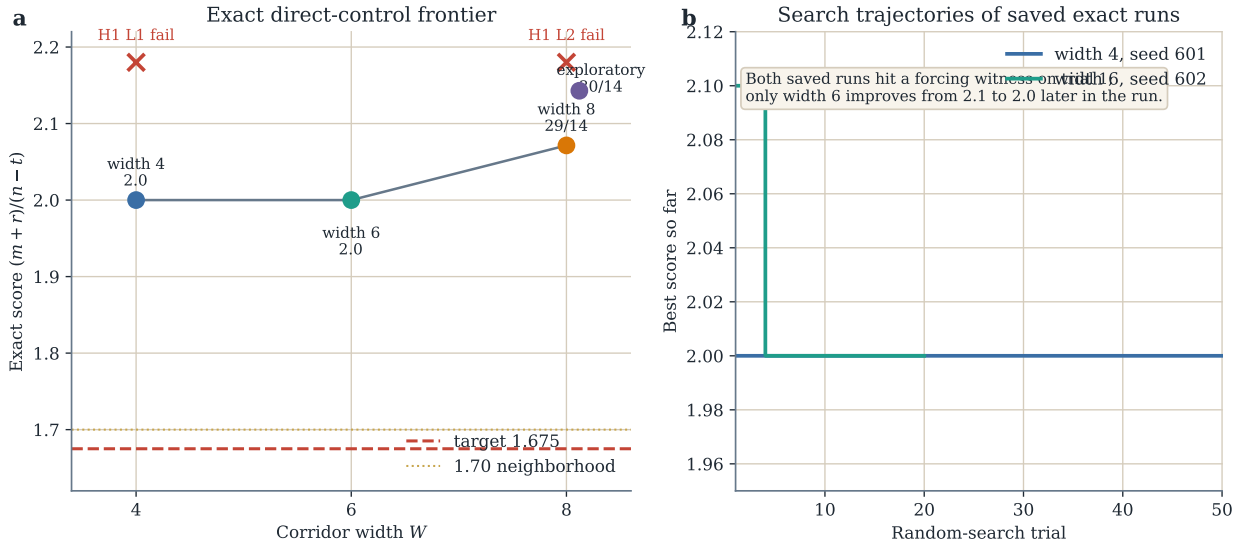

Figure 3: Exact frontier of the saved corridor program. **a**, the best saved direct-control scores at widths 4, 6, and exploratory width 8, together with the target threshold 1.675 and the broader 1.70 neighborhood used in the experiment plan. The two red crosses show the H1 level-1 and level-2 verification failures. **b**, best-score-so-far trajectories for the saved width-4 and width-6 runs. These are exact verifier outputs, not heuristic rollout scores [Archivara Research Lab, 2026i,j,k].

The explicit width-4 and width-6 certificates are useful because they show what the surviving positives actually look like. Figure 4 plots their corridor geometry, seeds, and initial solved vertices. Both witnesses are visibly boundary loaded. The width-6 witness even places two distinct singleton labels at the same rightmost top vertex, which is legal but underlines how small and hand-shaped the surviving controls are.

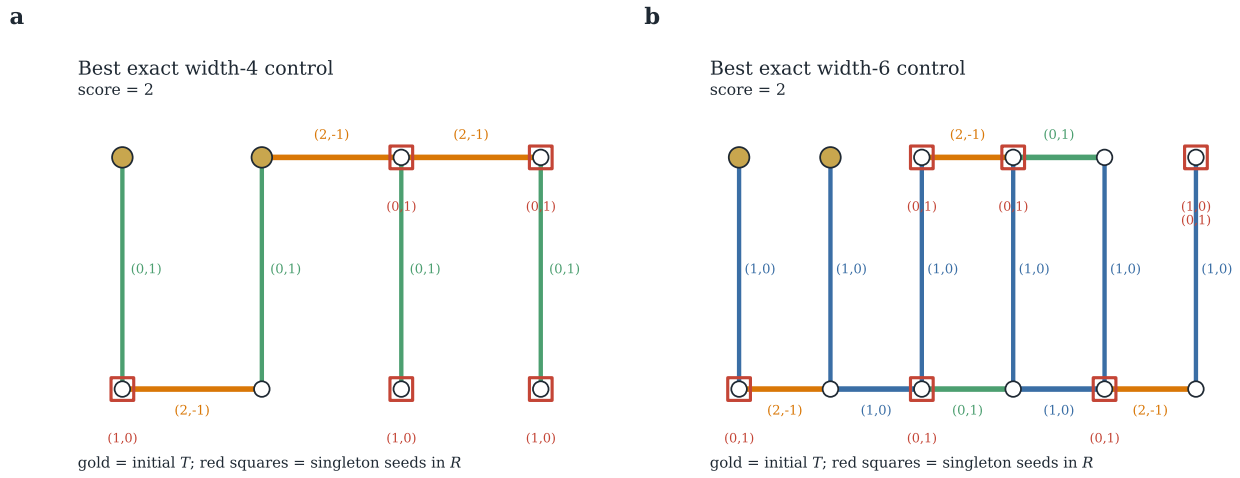

Figure 4: The best exact direct controls saved in the repository. **a**, the width-4 witness with score 2.0. **b**, the width-6 witness with score 2.0. Gold nodes mark the initial solved set T, red squares mark singleton seeds in R, and colored edges show nonzero labels. These diagrams are rendered directly from the saved JSON certificates [Archivara Research Lab, 2026i,j].

7.3 H2 Adds No Screening Value

The saved H2 experiment compared four family IDs: H1 level 1, H1 level 2, the width-4 direct control, and the width-6 direct control. Table 3 reports the exact features that mattered. The direct controls are already separated from the dead H1 rows by raw support size and target-solvable count alone. The additional abelian-style descriptors never change the decision boundary [Archivara Research Lab, 2026f].

Family	forcing?	r	rank	Smith tail	m/n	target-solvable
H1 level 1	no	1	8	1	0.875	0
H1 level 2	no	1	16	1	0.9375	0
width 4 control	yes	5	12	2	0.875	2
width 6 control	yes	7	20	2	1.0833	1

Table 3: H2 invariant summary on the saved family IDs. The determinant pattern and coordinate height are constant across all four rows and are omitted for brevity. Raw support size and target-solvable counts already separate the dead H1 rows from the surviving direct controls [Archivara Research Lab, 2026f].

Figure 5 makes the same failure visually explicit. Panel (a) presents the feature matrix used in the screen. Panel (b) shows that a two-dimensional projection using support size and target-solvable count already isolates the H1 failures. No additional ranking lift is visible. This empirical outcome matches the route design: H2 was only allowed to survive if the imported invariants demonstrably beat the direct arithmetic descriptors on the same family set. They did not.

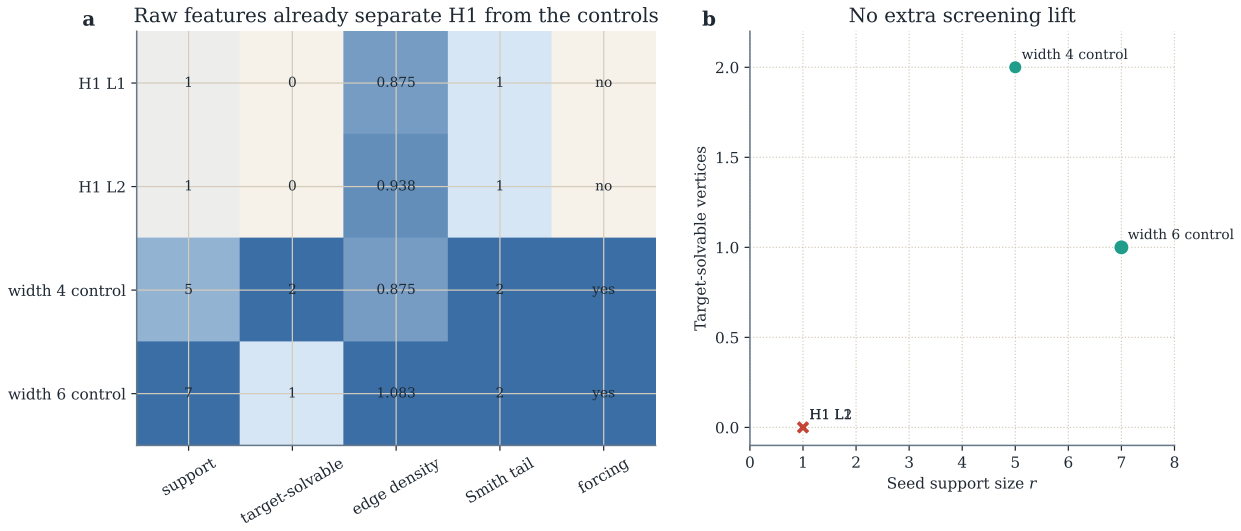

Figure 5: H2 screening results. **a**, the saved feature matrix for the four screened family IDs. **b**, a low-dimensional view of the same data: support size and target-solvable count already separate the dead H1 rows from the surviving direct controls. The point of the figure is not that these two features are universally sufficient, but that the imported H2 package adds no visible screening value on the tested family set [Archivara Research Lab, 2026f].

7.4 Ablations Show Boundary Sensitivity and Poor Transfer

The ablation study starts from the width-4 control with score 2.0. Only one nontrivial ablation remains successful: replacing the bottom-row labels by the top-row pattern preserves forcing but worsens the score to $13/6 \approx 2.1667$. Every other executed ablation fails exact verification or fails to recover a forcing witness under the fixed-motif search.

Ablation	Exact outcome
isotropic top $\to$ bottom	forcing, score $13/6 \approx 2.1667$
isotropic bottom $\to$ top	verification failure
boundary seed removal	verification failure
randomize R	$0/4$ successes
randomize T	$0/4$ successes
randomize X	$3/4$ successes, all at score 2.0
freeze X, scale to width 6	no forcing witness found
freeze X, scale to width 8	no forcing witness found

Table 4: Saved ablation outcomes for the width-4 direct control. The stage-order perturbation row was not applicable because no grammar-mediated H1 family survived [Archivara Research Lab, 2026a].

Figure 6 supplements Table 4 with uncertainty information for the randomized rows. The saved R- and T-randomization success rates are $0/4$, with 95% Wilson upper bounds of approximately 0.49. The saved X-randomization success rate is $3/4$, with a 95% Wilson interval of roughly $[0.30, 0.95]$. The width-8 boundary-stress archive contains $0/16$ successes, giving an upper Wilson bound of about 0.19. These are tiny samples, but they already support the qualitative claim that the surviving direct-control mechanism is strongly dependent on boundary placement and much less dependent on the exact arithmetic label identities than on the corridor geometry and seed layout.

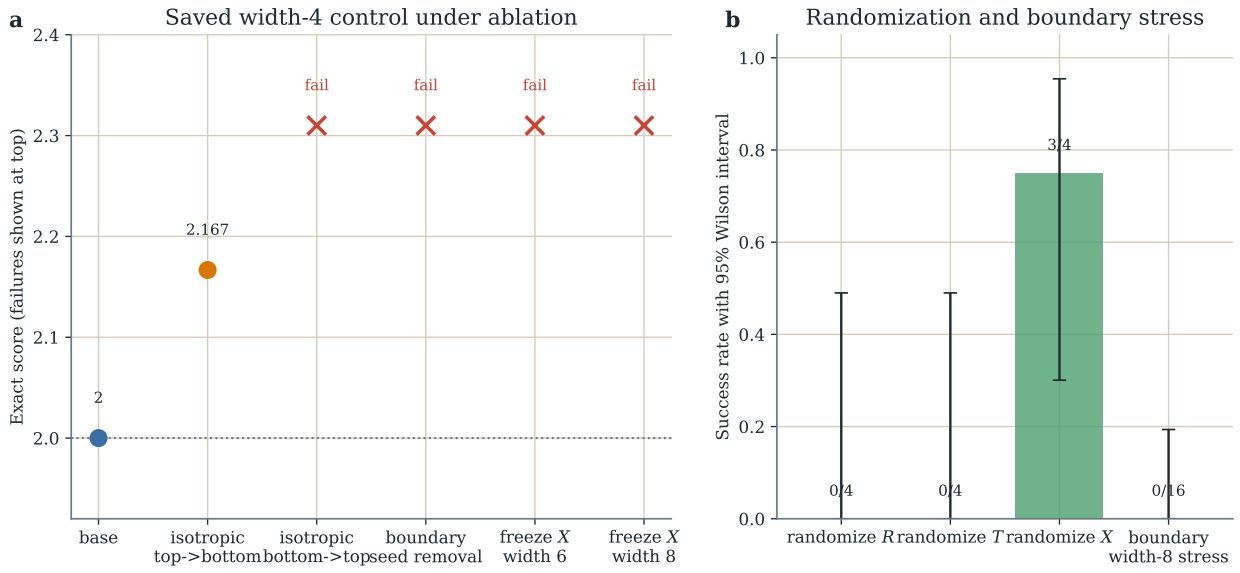

Figure 6: Saved ablation results. **a**, exact scores for successful ablations, with failures plotted at the top of the panel. Only the isotropic top-to-bottom replacement survives, and it worsens the score. **b**, success rates for the randomized and boundary-stress rows together with 95% Wilson intervals. These intervals are descriptive summaries of the saved finite samples rather than asymptotic statistical claims [Archivara Research Lab, 2026a].

8 Discussion

8.1 What the Repository Actually Established

The strongest claim supported by the saved artifacts is narrow and negative:

- the frozen cellular-automaton corridor route is killed exactly by Theorem 9;
- the backup abelian screening layer adds no visible value on the tested family IDs;
- the only surviving positive rows are small direct corridor controls around score 2.0.

This is still a meaningful result. The repository does not merely report that a heuristic search missed the target. It proves that one carefully specified local mechanism cannot work in the way initially hoped, and it does so with exact verifier-backed evidence rather than informal intuition. In automated mathematical research, such falsifications are valuable: they prevent future work from recycling a dead route under more optimistic language.

8.2 Limitations

The negative conclusion is rigorous for the frozen corridor program, but the benchmark section remains narrow. The verification memo identifies four gaps that we preserve rather than conceal [Archivara Research Lab, 2026h].

1. There is no fully matched width-8 direct baseline with the same four-nonzero palette and search budget as the width-4 and width-6 rows. The saved width-8 witnesses are exploratory only.
2. Several pre-registered non-CA baselines were never executed: low-height asymmetric X, explicit bounded-slope controls, and slowly growing-X schedules.
3. The stored JSON traces are not rich enough to support a benchmark claim about efficiency, hidden complexity, or search-budget frontiers beyond the narrow saved rows.
4. The entire study lives in a height-2 corridor regime. Nothing here rules out better certificates in broader constructible-graph families.

These limitations matter for interpretation. The paper does *not* show that cellular automata are useless for arithmetic Kakeya in general. It shows that one pre-registered corridor route fails exactly, and that the remaining direct controls in the same regime are weak and fragile.

8.3 Failure Modes and Open Questions

The theory and experiments point to three concrete failure modes.

One-seed bottlenecks. Theorem 9 shows that any route with one singleton seed and empty initial solved set is doomed, independently of geometry. Future local grammars must therefore create either multiple independent seed directions or nontrivial initial solved sets without simply hiding the complexity there.

Boundary programming. The saved positive witnesses lean heavily on boundary seeds and small initial solved sets. Removing or randomizing those structures kills the width-4 control immediately. Any future route must demonstrate that its gain is not merely boundary placement in disguise.

Frozen-X non-transfer. The direct controls do not improve with width under frozen-X scaling, and the saved frontier remains far above the target. This is consistent with the warning that small fixed alphabets can remain trapped in low-complexity basins [Tao, 2025], although the repository does not measure rational complexity directly and we therefore keep that comparison interpretive rather than factual.

Open questions follow directly.

1. Can one design a verifier-legal local grammar whose extracted instances necessarily contain at least two algebraically independent seed directions?
2. Does any corridor program with height greater than 2 evade the one-seed obstruction without moving all complexity into R or T?
3. What happens if the search space is widened from corridors to more general constructible graphs while retaining exact verification and matched direct baselines?
4. Are there direct-search families outside the corridor regime that beat score 2.0 without relying on growing X?

8.4 Societal and Epistemic Implications

The direct societal impact of this project is low. The broader implication is epistemic. Machine-assisted mathematical research is especially vulnerable to novelty inflation: a vivid metaphor, a local simulator, or an imported vocabulary can look like mathematical progress even when the exact verifier says otherwise. The repository's strongest positive contribution may therefore be procedural. It shows how to keep theorem-level claims, computational measurements, and local research artifacts on separate citation boundaries. That discipline is not cosmetic. It is part of how negative results become reusable scientific knowledge rather than abandoned search logs.

9 Conclusion

The frozen cellular-automaton corridor route to the arithmetic Kakeya benchmark does not work. The exact reason is now explicit. For the saved H1 pilot, every extracted instance has one singleton seed and empty initial solved set, so Theorem 9 rules out forcing before any score can be computed. The saved H2 backup adds no screening value on the tested family set. The best surviving exact objects are small direct corridor controls with score 2.0 at widths 4 and 6, together with worse exploratory width-8 witnesses. Ablations show that these controls are boundary-sensitive and do not scale cleanly under frozen X.

That is a negative result, but it is a sharp one. The paper leaves behind a sound-and-complete verifier for the corridor family, a clean obstruction theorem, explicit surviving control certificates, and an audit trail that prevents the dead cellular-automaton route from being rediscovered under softer language. For this repository state, that is the honest scientific endpoint.

A Explicit Certificates

The saved width-4 and width-6 witnesses are reproduced verbatim below in the six-line benchmark format.

A.1 Width 4 Control

```
12/6 m=7 r=5 n=8 t=2
[(0,0), (0,1), (1,0), (2,-1), (3,-2)]
[2,4]
[{1:(0,1)}, {(1,2):(2,-1), (1,3):(2,-1), (2,1):(2,-1)}]
[[1,1], [1,2]]
[{[2,1]:(1,0)}, {[1,3]:(0,1)}, {[1,4]:(0,1)}, {[2,3]:(1,0)}, {[2,4]:(1,0)}]
```

A.2 Width 6 Control

```
20/10 m=13 r=7 n=12 t=2
[(0,0), (0,1), (1,0), (2,-1), (3,-2)]
[2,6]
[{1:(1,0)}, {(1,3):(2,-1), (1,4):(0,1), (2,1):(2,-1), (2,2):(1,0),
(2,3):(0,1), (2,4):(1,0), (2,5):(2,-1)}]
```

```
[[1,1], [1,2]]
[{[2,1]:(0,1)}, {[1,3]:(0,1)}, {[1,4]:(0,1)}, {[2,3]:(0,1)}, {[2,5]:(0,1)},
 {[1,6]:(1,0)}, {[1,6]:(0,1)}]
```

A.3 Exploratory Width 8 Control

```
29/14 m=20 r=9 n=16 t=2
[(0,0), (0,1), (1,0), (2,-1)]
[2,8]
[{1:(2,-1)}, {(1,1):(1,0), (1,2):(0,1), (1,3):(1,0), (1,4):(1,0),
(1,5):(1,0), (1,6):(0,1), (1,7):(1,0), (2,1):(1,0), (2,2):(2,-1),
(2,3):(2,-1), (2,5):(1,0), (2,6):(2,-1)}]
[[1,1], [1,2]]
[{[1,6]:(0,1)}, {[2,1]:(0,1)}, {[1,5]:(0,1)}, {[2,5]:(0,1)},
 {[1,8]:(0,1)}, {[2,3]:(0,1)}, {[2,4]:(0,1)}, {[2,7]:(0,1)}, {[2,8]:(0,1)}]
```

B Additional Proof Commentary

This appendix records two small proof details used repeatedly in the main text.

Lemma 12. *If a vertex u is solvable from a solved set S and $S \subseteq S' \subseteq V \setminus \{u\}$, then u is also solvable from S'.*

Proof. By solvability from S, there exists a relation whose support outside S is exactly u. Because $S \subseteq S'$, every vertex outside S' is also outside S. Therefore the same relation has support outside S' contained in $\{u\}$. Since $u \notin S'$, that support is exactly u. So the same relation certifies solvability from S'. □

Lemma 13. *If $x \notin \langle(1,-1)\rangle$ and $k \in \mathbb{Z} \setminus \{0\}$, then $kx \notin \langle(1,-1)\rangle$.*

Proof. Suppose $kx = \lambda(1,-1)$ for some rational λ. Dividing by the nonzero integer k gives

$$x = \frac{\lambda}{k}(1,-1),$$

which contradicts $x \notin \langle(1,-1)\rangle$. □

C Reproducibility Details

C.1 Exact Commands

The saved markdown artifacts record the exact commands used to produce the core results. We reproduce the central ones here for completeness.

```
python3 scripts/ca_kakeya_search.py h1-obstruction --sum-value 1 --nonzero-count 4
```

```
python3 scripts/ca_kakeya_search.py random-search \
```

```
  --width-min 4 --width-max 4 --trials 50 --nonzero-count 4 \
  --seed-budget 8 --initial-t-budget 2 --allow-zero-horizontal \
  --rng-seed 601 --output results/phase4_width4_seed601.json

python3 scripts/ca_kakeya_search.py random-search \
  --width-min 6 --width-max 6 --trials 20 --nonzero-count 4 \
  --seed-budget 8 --initial-t-budget 2 --allow-zero-horizontal \
  --rng-seed 602 --output results/phase4_width6_seed602.json

python3 scripts/phase4_h2_screen.py \
  --control results/phase4_width4_seed601.json \
  --control results/phase4_width6_seed602.json \
  --output results/phase4_h2_screen.json

python3 scripts/phase4_ablations.py \
  --base results/phase4_width4_seed601.json \
  --output results/phase4_ablations.json
```

C.2 Figure Generation

All paper figures were generated from saved local JSON artifacts by `scripts/generate_paper_figures.py`. No figure depends on unpublished runs or on manual editing of the underlying numerical values.

References

Archivara Research Lab. Phase 4 ablation report, 2026a. Repository artifacts: `results/phase4_ablations.md` and `results/phase4_ablations.json`; recorded in commit 7316be00.

Archivara Research Lab. Phase 2 certificate grammar for the frozen corridor-substitution program, 2026b. Repository artifact: `results/phase2_certificate_grammar.md`.

Archivara Research Lab. Phase 4 h1 frontier note, 2026c. Repository artifact: `results/phase4_h1_frontier.md`; recorded in commit 53665011.

Archivara Research Lab. Exact one-seed obstruction for the frozen h1 corridor pilot, 2026d. Repository artifact: `results/phase4_h1_obstruction.json`; recorded in commit 53665011.

Archivara Research Lab. Phase 3 h1 program, 2026e. Repository artifact: `results/phase3_h1_program.md`.

Archivara Research Lab. Phase 4 h2 screening report, 2026f. Repository artifacts: `results/phase4_h2_screen.md` and `results/phase4_h2_screen.json`; recorded in commit 7316be00.

Archivara Research Lab. Exact search utilities for width-2 arithmetic kakeya corridor certificates, 2026g. Repository script: `scripts/ca_kakeya_search.py`; introduced in commit 2e939d91.

Archivara Research Lab. Verification summary for the cellular-automaton arithmetic kakeya program, 2026h. Repository artifact: `results/verification/verification_summary.md`.

Archivara Research Lab. Exact width-4 corridor control witness, 2026i. Repository artifact: `results/phase4_width4_seed601.json`; recorded in commit 53665011.

Archivara Research Lab. Exact width-6 corridor control witness, 2026j. Repository artifact: `results/phase4_width6_seed602.json`; recorded in commit 53665011.

Archivara Research Lab. Exploratory width-8 corridor witnesses, 2026k. Repository artifacts: `results/phase4_width8_seed501.json` and `results/phase4_sparse_unrestricted_seed502.json`; recorded in commit 53665011.

Bela Bollobas, Hugo Duminil-Copin, Robert Morris, and Paul Smith. Universality for two-dimensional critical cellular automata. *Proceedings of the London Mathematical Society*, 110(6): 1517–1569, 2015. doi: 10.1112/plms/pdv017. URL https://arxiv.org/abs/1406.6680.

Benjamin Bond and Lionel Levine. Abelian networks I. foundations and examples. *SIAM Journal on Discrete Mathematics*, 30(2):856–874, 2016a. doi: 10.1137/15M1030984. URL https://arxiv.org/abs/1309.3445.

Benjamin Bond and Lionel Levine. Abelian networks II: Halting on all inputs. *Selecta Mathematica*, 22 (1):319–340, 2016b. doi: 10.1007/s00029-015-0192-z. URL https://arxiv.org/abs/1409.0169.

Benjamin Bond and Lionel Levine. Abelian networks III: The critical group. *Journal of Algebraic Combinatorics*, 43(3):635–663, 2016c. doi: 10.1007/s10801-015-0648-4.

Charlie Cowen-Breen, Elene Karangozishvili, N. Varadarajan, and Thomas Wang. Pattern problems related to the arithmetic kakeya conjecture, 2020. URL https://arxiv.org/abs/2011.07056. arXiv:2011.07056.

Ben Green and Imre Ruzsa. On the arithmetic kakeya conjecture of katz and tao. *Periodica Mathematica Hungarica*, 79(2):147–168, 2019. doi: 10.1007/s10998-018-0270-z. URL https://arxiv.org/abs/1712.02108.

Ivailo Hartarsky and Tamas Mezei. Complexity of two-dimensional bootstrap percolation difficulty: Algorithm and NP-hardness. *SIAM Journal on Discrete Mathematics*, 34(2):1449–1483, 2020. doi: 10.1137/19M1239933.

Brett Hemenway, Rafail Ostrovsky, and Mary Wootters. Local correctability of expander codes. *Information and Computation*, 243:312–343, 2015. doi: 10.1016/j.ic.2014.12.013. URL https://arxiv.org/abs/1304.8129.

Nets Katz and Terence Tao. Bounds on arithmetic projections, and applications to the kakeya conjecture. *Mathematical Research Letters*, 6(5-6):625–630, 1999. doi: 10.4310/MRL.1999.V6.N6.A3.

Aleksander Kubica and John Preskill. Cellular-automaton decoders with provable thresholds for topological codes. *Physical Review Letters*, 123(2):020501, 2019. doi: 10.1103/PhysRevLett.123.020501. URL https://arxiv.org/abs/1809.10145.

Cristian Pohoata and Dmitrii Zakharov. Generalized arithmetic kakeya, 2024. URL https://arxiv.org/abs/2411.13395. arXiv:2411.13395.

Michael Sipser and Daniel A. Spielman. Expander codes. *IEEE Transactions on Information Theory*, 42(6):1710–1722, 1996. doi: 10.1109/18.556667.

Terence Tao. Sum-difference exponents for boundedly many slopes, and rational complexity, 2025. URL https://arxiv.org/abs/2511.15135. arXiv:2511.15135.